CP 312, Fall 2025
Assignment 4 (6% of the final grade)
(due Monday, November 17, at 23:30)

There are four questions in this assignment.

1. [10 marks] **Dynamic Programming.** The *longest exponential subsequence* of a sequence of n positive integers $a_1, a_2, \ldots, a_n$ is the longest subsequence such that each integer in the subsequence is at least twice as large as the previous one.

   For example, given the instance
   `1, 256, 16, 4096, 4, 1024, 64, 16384, 2, 128, 32, 8192, 8, 2048, 512, 32768`
   one of the longest exponential subsequences is `1, 16, 64, 128, 2048, 32768`.
   (Note that there are multiple exponential subsequences of length 6 for this example.)
   Design an algorithm that finds a longest exponential subsequence in time $O(n^2)$:

   (a) Give a precise definition of a subproblem that can be used to solve your problem with a dynamic programming algorithm.

   (b) Describe and justify the recurrence rule that will be used to compute the solutions to the subproblems defined in part (a).

   (c) Provide a pseudocode for your dynamic programming algorithm that solves your problem using the subproblems and recurrence rule defined above. Your algorithm needs to find a longest exponential subsequence: the answer is length of the longest exponential subsequence and also list of elements forming it. For the input above possible output is:

   ```
   length: 6
   subsequence: 1, 16, 64, 128, 2048, 32768
   ```

   (d) Analyze the time complexity of the algorithm obtained in part (c).

2. [7 marks] **Dynamic Programming.** Consider a variation of the Knapsack problem. There are two knapsacks that have capacity $W_1 > 0$ and $W_2 > 0$, respectively. There are n items $1, 2, \ldots, n$. Item i has weight $w(i) > 0$ and two values $v_1(i) > 0$ and $v_2(i) > 0$. Here $v_k(i)$ is the value one gains by putting item i into knapsack k ($k = 1, 2$). Note, that values $v_1(i)$ and $v_2(i)$ are generally speaking different. The "Two Knapsacks Problem" is to find two *disjoint* subsets of items S_1 and S_2, such that

   1. $\sum_{i \in S_1} w(i) \leq W_1$,
   2. $\sum_{i \in S_2} w(i) \leq W_2$, and
   3. $V = \sum_{i \in S_1} v_1(i) + \sum_{i \in S_2} v_2(i)$ is maximized.

   Give a dynamic programming algorithm to find the maximum value V. Your algorithm does not need to find the sets S_1 and S_2. Clearly indicate what your subproblems are, and the order in which you solve them. Justify correctness of your algorithm, and analyze its running time. Is your algorithm a polynomial-time algorithm? Why or why not?

3. [7 marks] The *distance* between two vertices in a connected undirected graph is the length of the shortest simple path between them (number of edges on the shortest simple path). The *diameter* of a connected undirected graph is the maximum distance between vertices in the graph. Give an efficient algorithm to compute the diameter of a graph. Provide justification of the correctness and a its running time complexity in terms of $|V|$ and $|E|$.

4. [6 marks] Solve previous problem in simplified settings: given graph is a binary tree. The *diameter* of a tree is the maximum distance between vertices in the tree. Solve this problem using divide and conquer approach. Your algorithm takes a pointer to the root of a binary tree as a parameter, and returns the diameter.

   Is this algorithm asymptotically faster than algorithm in question 3?

Submission.

Submit single pdf file `A4.pdf` to the assignment 4 dropbox on `MyLearningSpace`.